How ICE Is Injected Into the Model

A step-by-step explanation with formulas

Phung Gia Huy

October 2026

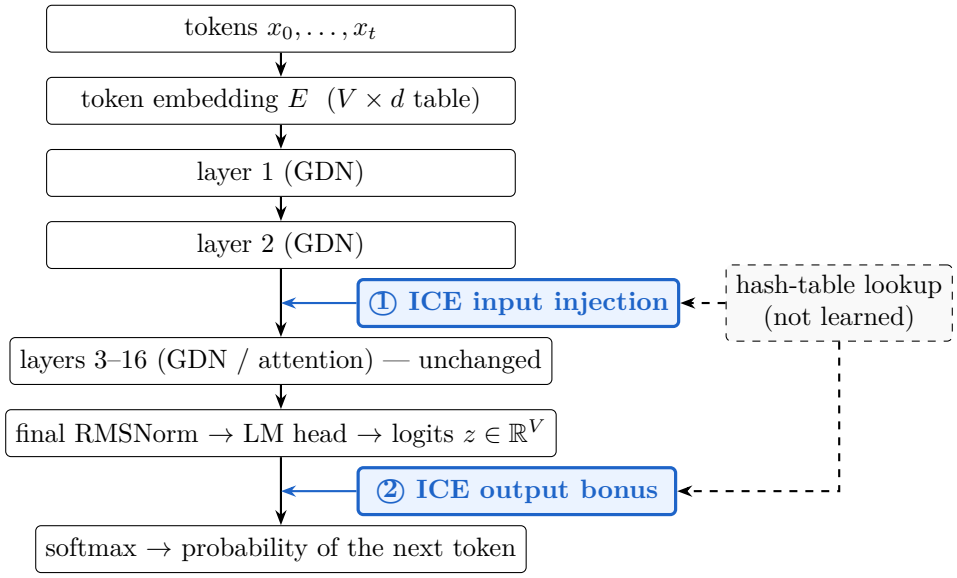
The whole idea in three sentences

A **fixed (non-learned) lookup** finds where the last n tokens appeared earlier and reports the token that came next. The model **learns how much to trust** that hint, using two small learned modules: one inside the network and one at the output. **ICE-dropout** sometimes hides the hint during training, so attention does not become lazy.

Throughout, we use the model from the proposal: a hybrid with 16 layers, hidden size $d = 768$ and vocabulary size $V \approx 32{,}000$.

1 Where ICE plugs into the model

ICE touches the model in exactly **two places**. Everything else is a normal hybrid model.



- ① **Input injection**, after layer 2: the hints are added to the hidden state, so layers 3–16 can use them.
- ② **Output bonus**, at the logits: the copied token gets extra score.

2 Step 0: what the lookup returns (no learning)

At position t the model must predict x_{t+1}. For each order $n \in \{1, 2, 3, 4, 6, 8\}$, the hash table looks for the most recent earlier position $p < t$ where the same last n tokens appeared, and returns four integers:

Symbol	Meaning
$m_n \in \{0, 1\}$	1 if a match exists, 0 if not
s_n	the *successor*: the token that came after the earlier match, x_{p+1}
ℓ_n	the match length (as a bucket)
δ_n	how far back the match is: $\lfloor \log_2(t - p) \rfloor$

Example

Text so far: `... user = get_user(req, id) ... user = get_user(req,` $\rightarrow$?

order n	match? m_n	successor s_n	distance
8	1	`id`	512 tokens back
4	1	`id`	512 tokens back
1	1	`self`	3 tokens back (another call)

The long matches are reliable; the 1-token match is noise. The model must learn this.

Because $p < t$, the successor x_{p+1} is always a token the model has *already seen*. ICE never looks at the future.

3 Injection ①: adding the hints inside the network

Let $h_t \in \mathbb{R}^{768}$ be the hidden state of token t after layer 2.

Step 1 — turn each hint into a vector

$$\boxed{f_n \;=\; W_n\, E[s_n] \;+\; L[\ell_n] \;+\; D[\delta_n]}$$

- $E[s_n] \in \mathbb{R}^{768}$: the embedding of the hinted token. It is the **same table** the model uses for its input words, so the hint "speaks the model's language" from the start.
- $W_n \in \mathbb{R}^{768 \times 768}$: a learned matrix, one per order. An 8-gram hint and a 1-gram hint can carry different messages.
- $L[\ell_n]$, $D[\delta_n] \in \mathbb{R}^{768}$: small learned vectors for "how long the match was" and "how far back it was".

In words, f_n says: *"the hint is token* `id`*, from an 8-token match, 512 tokens back."*

Step 2 — decide how much to trust it (the gate)

$$\boxed{g_n \;=\; \sigma\!\left(\frac{\langle \mathrm{RMSNorm}(h_t),\ \mathrm{RMSNorm}(f_n)\rangle}{\sqrt{d}}\right) \in (0,1)}$$

- The dot product $\langle\cdot,\cdot\rangle$ measures **how well the hint agrees with what the model currently thinks**.
- If the context suggests "we are inside a repeated function call", h_t and f_n point in similar directions, so the dot product is large and $g_n \approx 1$.
- If the hint makes no sense here, they disagree and $g_n \approx 0$.
- σ (the sigmoid) squashes the result into $(0,1)$, like a **volume knob**.
- RMSNorm puts both vectors on the same scale, so the gate depends on *direction* (agreement), not on size.

Step 3 — add the gated hints to the hidden state

$$\boxed{h_t \;\leftarrow\; h_t \;+\; \sum_n m_n\, g_n\, f_n}$$

Orders with no match have $m_n = 0$ and add nothing. Layers 3–16 then run **unchanged** on the enriched h_t: they can use the hint, combine it with what attention finds, or ignore it.

Example (continued)

Suppose the learned gates give $g_8 = 0.9$, $g_4 = 0.8$, $g_1 = 0.05$. Then

$$h_t \leftarrow h_t + 0.9\, f_8 + 0.8\, f_4 + 0.05\, f_1,$$

so the model mostly "hears" the reliable 8-gram and 4-gram hints and almost ignores the 1-gram hint.

4 Injection ②: the copy bonus at the output

After the last layer, the model computes its normal logits (one score per vocabulary token):

$$z \;=\; W_{\text{out}}\,\mathrm{RMSNorm}(h_t^{(16)}) \;\in\; \mathbb{R}^V.$$

Let s^* be the successor from the **longest** match (here `id`). ICE adds a learned bonus to that single entry:

$$\boxed{\lambda_t = \mathrm{softplus}\Big(w^\top \mathrm{RMSNorm}(h_t^{(16)}) + b[\ell^*]\Big) \;\geq 0, \qquad z[s^*] \;\leftarrow\; z[s^*] + \lambda_t}$$

- $w \in \mathbb{R}^{768}$ and b (one number per match-length bucket) are learned.
- $\mathrm{softplus}(u) = \log(1 + e^u)$ is always ≥ 0: ICE can only *push up* the hinted token, never push it down.

- λ_t is the model's **copy confidence**, recomputed at every token.
- Initialization: $w = 0$, $b = -2$, so $\lambda_t \approx 0.13$ at the start and training begins close to a model without ICE.

The next-token probability is then the usual $p = \text{softmax}(z)$.

Numeric example

token	logit (no ICE)	prob. (no ICE)	logit with $\lambda_t = 2$	prob. with ICE
`id`	3.0	0.42	**5.0**	**0.84**
`user_id`	2.8	0.34	2.8	0.09
`name`	2.0	0.15	2.0	0.04
others	—	0.09	—	0.02

With a confident bonus ($\lambda_t = 2$), the probability of `id` doubles. With low confidence ($\lambda_t \approx 0$), nothing changes.

5 How ICE learns: it is just the normal training loss

ICE needs **no extra loss and no labels**. The model is trained with ordinary next-token prediction:

$$\mathcal{L} = -\log p(x_{t+1}).$$

The copy bonus (cleanest case)

For a softmax, the gradient of the loss with respect to the bonus is

$$\boxed{\frac{\partial \mathcal{L}}{\partial \lambda_t} = p(s^*) - \mathbf{1}[\, s^* = x_{t+1}\,]}$$

Situation	Gradient	What training does
Hint correct ($s^* = x_{t+1}$)	$p(s^*) - 1 < 0$	**increases** λ here: "trust more"
Hint wrong ($s^* \neq x_{t+1}$)	$p(s^*) > 0$	**decreases** λ here: "trust less"

Because λ_t is computed from the hidden state h_t and the match length, the updates to w and b teach the model *in which situations* to trust the hint: long matches, code, repeated function calls, and so on.

The input gate

By the chain rule,

$$\frac{\partial \mathcal{L}}{\partial g_n} = \Big\langle \frac{\partial \mathcal{L}}{\partial h_t},\ m_n f_n \Big\rangle.$$

If adding the hint f_n moves h_t in a direction that lowers the loss, the gate opens more; otherwise it closes. The matrices W_n, the tables L and D, and the norms are updated at the same time, so the hint vectors themselves become more useful.

What the model ends up learning (nobody writes these rules)

- "An 8-token match inside code is almost always right" $\Rightarrow$ large g_8 and large λ.
- "A 1-token match is nearly random" $\Rightarrow$ $g_1 \approx 0$.
- "A match right after `range(` is unreliable" $\Rightarrow$ small λ there.

6 ICE-dropout

The problem: "lookup laziness"

If the hint is always available, many tokens can be predicted just by copying it. The full-attention layers then get little reason to learn *retrieval*. But ICE only handles **exact** repeats. When the text says the same thing in different words ("the blue door's code is 4721" ... "what number opens the door that is blue?"), the hint is useless, and a model whose attention never learned retrieval fails.

This is like a student who always does homework with the answer key: high scores in practice, failure in the exam.

The fix, as a formula

For each training **sequence**, flip one coin $k \sim \text{Bernoulli}(0.8)$: $k = 1$ (ICE on) 80% of the time, $k = 0$ (ICE off) 20% of the time. Multiply both injections by k:

$$h_t \leftarrow h_t + k \sum_n m_n \, g_n \, f_n, \qquad z[s^*] \leftarrow z[s^*] + k \, \lambda_t$$

- When $k = 0$, the model is a plain hybrid for that whole text, so attention *must* find the information itself and keeps learning retrieval.
- **At inference, $k = 1$ always.**
- The coin is per **sequence**, not per token, so attention must do the job across an entire text.
- Normal dropout rescales by $1/(1-p)$; we do not, because the gates already learn how loud the hint should be.

7 Code sketch (PyTorch)

```
class ICEInput(nn.Module):                          # injection 1, after layer 2
    def __init__(self, d, n_orders=6, n_len=8, n_dist=16):
        super().__init__()
        self.W = nn.ModuleList([nn.Linear(d, d, bias=False) for _ in range(n_orders)])
        self.L, self.D = nn.Embedding(n_len, d), nn.Embedding(n_dist, d)
        self.norm_h, self.norm_f = nn.RMSNorm(d), nn.RMSNorm(d)
```

```
    def forward(self, h, tok_emb, succ, len_b, dist_b, k):
        # h: [B,T,d]; succ/len_b/dist_b: [B,T,N] ints (succ = -1 if no match); k: [B] in {0,1}
        m = (succ >= 0).float()                                  # m_n
        e = tok_emb(succ.clamp(min=0))                           # E[s_n]
        f = torch.stack([self.W[n](e[:, :, n]) for n in range(len(self.W))], dim=2)
        f = f + self.L(len_b) + self.D(dist_b)                   # f_n
        g = torch.sigmoid((self.norm_h(h).unsqueeze(2) * self.norm_f(f)).sum(-1)
                          / h.shape[-1] ** 0.5)                  # g_n
        add = ((m * g).unsqueeze(-1) * f).sum(dim=2)             # sum_n m_n g_n f_n
        return h + k[:, None, None] * add                        # ICE-dropout via k

class ICEOutput(nn.Module):                          # injection 2, at the logits
    def __init__(self, d, n_len=8):
        super().__init__()
        self.w, self.b, self.norm = nn.Linear(d, 1), nn.Embedding(n_len, 1), nn.RMSNorm(d)
        nn.init.zeros_(self.w.weight); nn.init.constant_(self.b.weight, -2.0)

    def forward(self, logits, h_last, s_star, len_star, k):
        lam = F.softplus(self.w(self.norm(h_last)) + self.b(len_star.clamp(min=0))).squeeze(-1)
        lam = lam * (s_star >= 0) * k[:, None]                   # no match or dropped -> 0
        return logits.scatter_add(-1, s_star.clamp(min=0).unsqueeze(-1), lam.unsqueeze(-1))
```

In the model's forward pass: `h = ice_in(h, embed, ...)` right after layer 2, and `logits = ice_out(logits, h_final, ...)` right before the loss.

8 Summary

Piece	Formula	Learned?	Job
Lookup	$m_n, s_n, \ell_n, \delta_n$ from a hash table	no	find exact repeats
Hint vector	$f_n = W_n E[s_n] + L[\ell_n] + D[\delta_n]$	yes	describe the hint
Gate	$g_n = \sigma\big(\langle \mathrm{RMSNorm}\, h_t, \mathrm{RMSNorm}\, f_n\rangle / \sqrt{d}\big)$	yes	how much to listen
Input injection	$h_t \leftarrow h_t + k \sum_n m_n g_n f_n$	—	give hints to layers 3–16
Copy bonus	$z[s^*] \mathrel{+}= k\,\mathrm{softplus}(w^\top h + b[\ell^*])$	yes	push up the copied token
ICE-dropout	$k \sim \mathrm{Bernoulli}(0.8)$; $k = 1$ at test time	—	keep attention learning

Extra parameters: about 4M (mostly the six 768×768 matrices W_n), roughly 3% of the 140M model. **Extra memory at inference:** about 100 bytes per token (token IDs and hash-table entries), versus 2–3 KB per token for one full-attention layer.